

Descripción de los *point distribution models*

En este entregable vamos a considerar una aplicación de la descomposición en autovalores y autovectores (*eigenvalue decomposition* o EVD). Concretamente, vamos a usar la EVD para el análisis de componentes principales (*principal component analysis* o PCA) y su aplicación a los *point distribution models* (PDMs) en un ejemplo de modelado de caras.

Los PDMs describen el contorno promedio de objetos y su variabilidad a partir de un conjunto de datos de entrenamiento. El proceso consta de 5 pasos:

1. Crear un modelo a partir de una serie de puntos de referencia. Para el conjunto de datos que se va a considerar en este entregable, la Figura 1 muestra un ejemplo, que incluye la imagen de una cara y los contornos formados por los puntos de referencia (cejas, ojos, nariz, boca y mandíbula). El programa `transformar_BBDD.py` descarga la base de datos de la URL http://datasets.openimaj.org/imm_face_db.zip, lee las imágenes y las coordenadas de los puntos de referencia y lo guarda en un fichero `.npz`. Concretamente, la variable `structs` es un array con $N = 240$ elementos (uno por cada cara del conjunto de entrenamiento), siendo cada elemento un diccionario con los siguiente campos:
 - `landmarks`: es una lista de Numpy arrays, y cada array contiene las coordenadas de algún contorno (ojo izquierdo, mandíbula, ...). `structs[index]['landmarks'][index2]` accede a las coordenadas del contorno `index2` de la cara `index`.
 - `closed_path`: es una lista de booleanos que indica si el contorno está cerrado o no. Por ejemplo, para los ojos debería ser `true` y para la mandíbula `false`.
 - `image`: es un Numpy array que se corresponde con la imagen de la cara.
 - `resolution`: es una tupla que contiene la resolución de la imagen. Se podría obtener también como `structs[index]['image'].shape`.

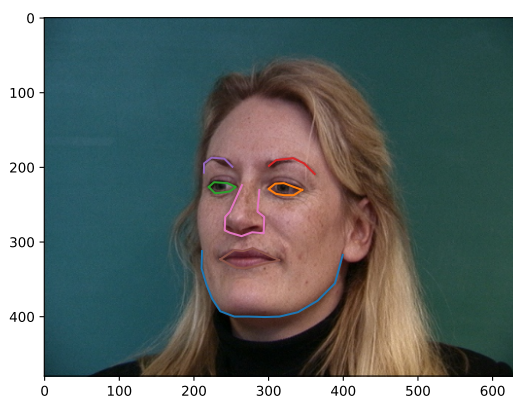


Figura 1: Ejemplo de cara y sus correspondientes puntos de referencia.

2. Alinear el conjunto de datos de entrenamiento en un eje de coordenadas común. Este alineamiento se hace mediante un algoritmo iterativo. La Figura 2 muestra todos los puntos de referencia de todos los ejemplos de entrenamiento sin alinear y alineados. Como se puede

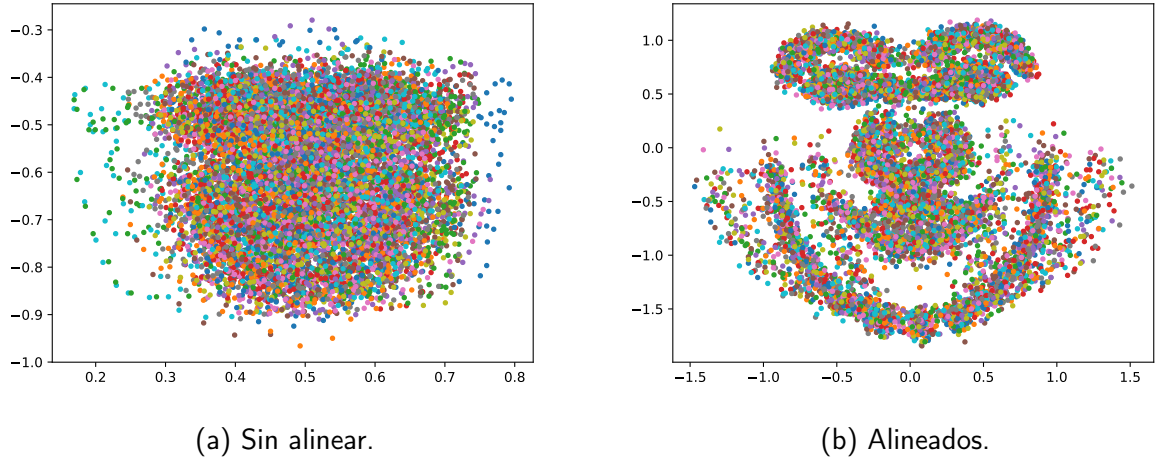


Figura 2: Puntos de referencia para todos los ejemplos de entrenamiento.

observar, cuando se alinean los puntos de referencia es posible inferir ciertas formas (mandíbulas más o menos anchas, por ejemplo). El programa `Alinear_landmarks.py` pinta en primer lugar una cara aleatoria junto con los contornos y, a continuación, alinea los puntos de referencia, guardando estos puntos en el campo `landmarks_aligned`. A diferencia de `landmarks`, `landmarks_aligned` es un único array con todos los puntos de referencia y tiene dos columnas: una con las coordenadas x y otra con las y de los puntos de referencia.

3. Obtener la matriz de covarianza muestral de los datos. Como el array `landmarks_aligned` tiene n puntos de referencia, $(x_1, y_1), \dots, (x_n, y_n)$, el primer paso es transformarlos en un vector $2n$ -dimensional: $\mathbf{z} = [x_1 \ \dots \ x_n \ y_1 \ \dots \ y_n]^T$. Concretamente, tendremos N vectores, $\mathbf{z}_1, \dots, \mathbf{z}_N$, uno por cada cara del conjunto de entrenamiento. Por lo tanto, la matriz de covarianza muestral viene dada por

$$\hat{\mathbf{R}} = \frac{1}{N} \sum_{i=1}^N (\mathbf{z}_i - \hat{\boldsymbol{\mu}}) (\mathbf{z}_i - \hat{\boldsymbol{\mu}})^T,$$

donde la media muestral es

$$\hat{\boldsymbol{\mu}} = \frac{1}{N} \sum_{i=1}^N \mathbf{z}_i.$$

4. Usamos la EVD para extraer las componentes (direcciones) principales

$$\hat{\mathbf{R}} = \mathbf{U} \boldsymbol{\Lambda} \mathbf{U}^T.$$

5. Generamos instancias de nuestro modelo

$$\tilde{\mathbf{z}} = \hat{\boldsymbol{\mu}} + \mathbf{U} \mathbf{b},$$

donde $\mathbf{b}$ controla los parámetros de los contornos. En este contexto, cada una de las columnas de $\mathbf{U}$ es una componente principal (o “modo”) y lo habitual es considerar unos pocos modos asociados a los mayores autovalores.

Problema 1

El objetivo de este entregable es generar 9 instancias del modelo para los tres modos principales. Es decir, para el primer modo, el vector de parámetros debería ser $\mathbf{b} = [b_1 \ 0 \ \cdots \ 0]^T$, suponiendo que la primera columna de $\mathbf{U}$ corresponde con el mayor autovalor. Para el segundo y tercer modos, el vector de parámetros debería ser $\mathbf{b} = [0 \ b_2 \ 0 \ \cdots \ 0]^T$ y $\mathbf{b} = [0 \ 0 \ b_3 \ 0 \ \cdots \ 0]^T$, respectivamente. Los valores que hay que usar para cada modo son $b_i = \{-3\sqrt{\lambda_i}, 0, 3\sqrt{\lambda_i}\}$, donde λ_i es el i -ésimo autovalor. El resultado final que habría que generar sería una matriz 3×3 de caras, donde cada fila corresponde a un modo y cada columna corresponde a cada uno de los tres valores de b_i , de manera similar a lo mostrado en la Figura 3. En esta figura se puede observar que la columna central muestra siempre la misma cara, ya que $b_i = 0$ en los tres casos, y se corresponde con el contorno promedio. Asimismo, debería dibujar en otra figura los autovalores y sobre la misma figura la suma acumulativa de los mismos.

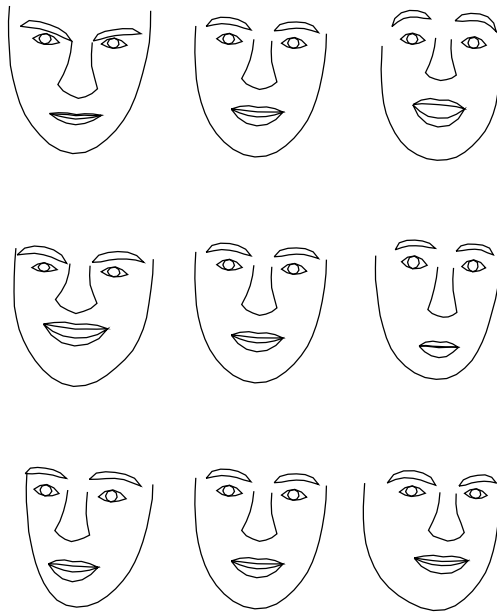


Figura 3: Ejemplo de contornos generados (Fuente: Cootes'00).

..... — — — — —